

CS 5600: Advanced Database Systems

Final Report — AI-Assisted Pet Allergy Symptom Tracking System

Student: Sam Cashion **Student ID:** 700619652 **Email:** scc96520@ucmo.edu **CRN:** 23405 | **Group:** Group 2

Course: CS 5600 — Advanced Database Systems, Spring 2026

Table of Contents

1. [Project Description](#)
 2. [Database Description](#)
 3. [Database Diagram](#)
 4. [Data Dictionary](#)
 5. [Sample Data](#)
 6. [User Interfaces and Forms](#)
 7. [Source Code](#)
 8. [NoSQL Commands](#)
-

1. Project Description

Purpose of the Database

The purpose of this database application is to track and analyze allergy-related symptoms in pets over time. This project is motivated by my personal experience managing chronic allergies in my own dog, Finn. Finn experiences recurring allergy symptoms such as severe itching and skin irritation that fluctuate daily. Without a structured way to record this information, it is difficult to identify patterns, evaluate symptom severity over time, or clearly summarize his allergy history.

The database provides a consistent and organized method for recording allergy symptoms and transforming daily observations into meaningful information. The system allows users to create pet profiles and log daily allergy symptoms, including the symptom type, severity level, date, and optional notes. This data enables the system to support queries that analyze symptom trends and summarize allergy activity over a selected period.

Users and Information Needs

The primary users of the system are pet owners managing pets with chronic or recurring allergies. These users need to understand:

- What symptoms their pet has experienced and how frequently
- Whether symptom severity is improving or worsening over time
- Which treatments are most effective at reducing symptoms
- How environmental factors (pollen, humidity, mold) correlate with symptom flare-ups

Problems the System Solves

- **Fragmented tracking** — pet owners rely on memory or scattered notes; this system provides a single structured record
- **Pattern identification** — without aggregated data, it is difficult to see that symptoms spike when pollen exceeds a certain threshold
- **Treatment effectiveness** — correlating treatment logs with symptom logs reveals whether medications like Apoquel or Cytopoint are working
- **Communication with veterinarians** — a structured history makes vet visits more productive

Input Data

- Pet profile information (name, species, breed, date of birth)
- Daily symptom entries: symptom type, severity rating (1–10), timestamp, optional notes
- Treatment administration records: medication/therapy type, dosage, timestamp
- Environmental factor readings: pollen count, temperature, humidity, mold spore count

LLM Integration

To improve accessibility, the system integrates a Large Language Model (LLM) using Ollama (Llama 3.2, running locally). Users can ask natural language questions such as:

- *"What are Finn's most common symptoms?"*
- *"Is there a correlation between pollen count and itching severity?"*
- *"Which treatments helped reduce symptoms the most?"*
- *"Show average severity by month."*

The LLM translates these questions into MongoDB aggregation pipelines, executes them against the live database, and returns a plain-English summary alongside the raw result data. All queries and responses are logged in the `llmQueryLogs` collection for audit purposes.

2. Database Description

Technology Stack

Layer	Technology	Version
Database	MongoDB (NoSQL Document Store)	8.3
Backend	Java + Javalin + MongoDB Driver	Java 17, Javalin 6.7.0, mongodb-driver-sync 5.2.0
Frontend	HTML + Vanilla JavaScript + Tailwind CSS	No build step required
LLM	Ollama (Llama 3.2, local)	REST API at localhost:11434
Build	Apache Maven (Shade plugin for fat JAR)	3.9+

Database Overview

The application uses MongoDB with **10 collections** organized into two groups:

Reference Collections (normalized, rarely change):

- `users`, `pets`, `symptomTypes`, `treatments`, `envFactorTypes`
- Log Collections** (time-series, write-heavy):
- `symptomLogs`, `treatmentLogs`, `envFactorLogs`, `llmQueryLogs`

Supporting Collection:

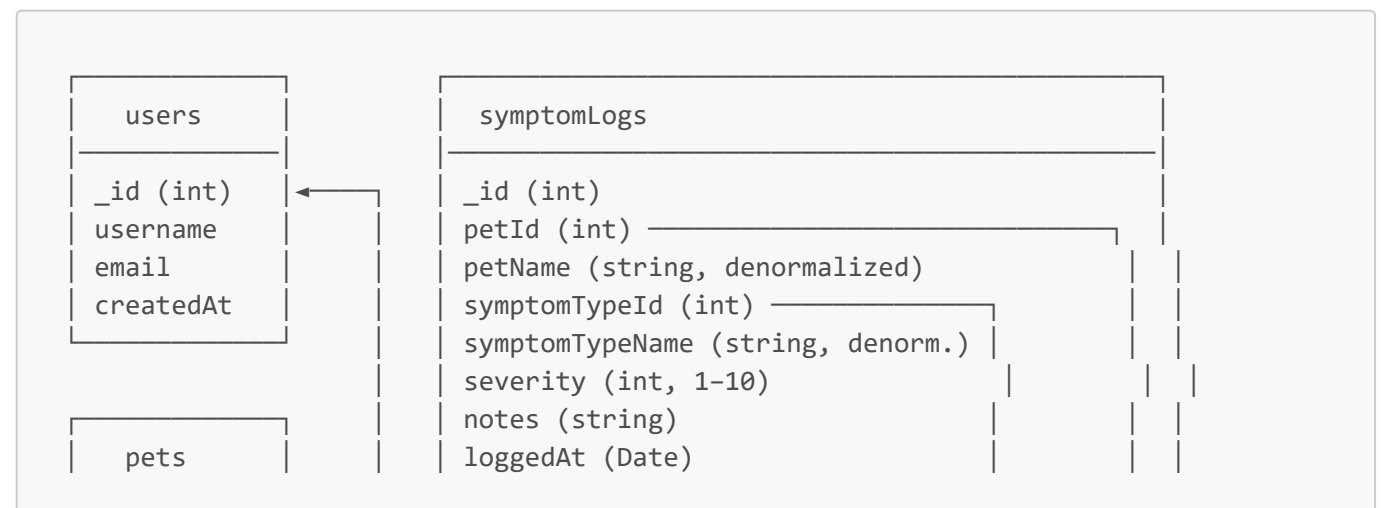
- `counters` — provides atomic auto-increment integer `_id` values for all collections

Key Design Decisions

- 1. Denormalized Log Documents** Log collections (`symptomLogs`, `treatmentLogs`, `envFactorLogs`) embed the pet name and type names at write time. For example, a `symptomLogs` document stores both `symptomTypeId: 1` and `symptomTypeName: "Itching"`. This eliminates `$lookup` joins in aggregation queries, making LLM-generated pipelines simpler, faster, and less error-prone.
- 2. Normalized Reference Collections** Reference data (`symptomTypes`, `treatments`, `envFactorTypes`) is kept in separate collections so that a name change (e.g., renaming a treatment) only requires updating one document rather than a bulk update across thousands of log entries.
- 3. Integer `_id` via Counters Collection** A `counters` collection provides atomic auto-increment using `findOneAndUpdate` with `$inc` and `ReturnDocument.AFTER`. This gives integer primary keys that are intuitive for a class demo and compatible with REST API routes (`/api/pets/1`).
- 4. Compound Indexes on Primary Query Patterns** All log collections are indexed on (`petId`, `loggedAt`) or (`petId`, `administeredAt`), matching the primary access pattern: "get all logs for a specific pet within a date range."
- 5. LLM Query Safety** The `OllamaService` validates every LLM-generated pipeline before execution:
 - Parses with `BsonDocument.parse()` — rejects invalid JSON
 - Validates the collection name against an explicit allowlist
 - Iterates pipeline stages to block `$out` and `$merge` (write stages)

3. Database Diagram

Collection Relationship Diagram





Indexes

Collection	Index Fields	Purpose
symptomLogs	(petId, loggedAt)	Date-range queries per pet
treatmentLogs	(petId, administeredAt)	Date-range queries per pet
envFactorLogs	(petId, loggedAt)	Date-range queries per pet

Collection	Index Fields	Purpose
<code>pets</code>	<code>(userId)</code>	Look up all pets for a user
<code>llmQueryLogs</code>	<code>(userId, createdAt DESC)</code>	Chat history, newest first

4. Data Dictionary

Collection: `users`

Field	Type	Required	Description
<code>_id</code>	int	Yes	Auto-incremented primary key
<code>username</code>	string	Yes	Unique login name
<code>email</code>	string	Yes	Unique email address
<code>createdAt</code>	Date	Yes	UTC timestamp of account creation

Collection: `pets`

Field	Type	Required	Description
<code>_id</code>	int	Yes	Auto-incremented primary key
<code>userId</code>	int	Yes	Reference to <code>users._id</code>
<code>name</code>	string	Yes	Pet's name (e.g., "Finn")
<code>species</code>	string	Yes	Species (e.g., "Dog", "Cat")
<code>breed</code>	string	No	Breed (e.g., "Golden Retriever")
<code>dateOfBirth</code>	Date	No	Pet's date of birth (UTC midnight)
<code>createdAt</code>	Date	Yes	UTC timestamp of record creation

Collection: `symptomTypes`

Field	Type	Required	Description
<code>_id</code>	int	Yes	Auto-incremented primary key
<code>name</code>	string	Yes	Unique symptom name (e.g., "Itching")
<code>description</code>	string	No	Plain-English description of the symptom

Seeded values: Itching, Skin Redness, Sneezing, Ear Inflammation, Paw Licking, Watery Eyes, Nasal Discharge

Collection: `symptomLogs`

Field	Type	Required	Description
<code>_id</code>	int	Yes	Auto-incremented primary key
<code>petId</code>	int	Yes	Reference to <code>pets._id</code>
<code>petName</code>	string	Yes	Denormalized pet name (embedded at write time)
<code>symptomTypeId</code>	int	Yes	Reference to <code>symptomTypes._id</code>
<code>symptomTypeName</code>	string	Yes	Denormalized symptom name (embedded at write time)
<code>severity</code>	int	Yes	Severity score 1 (mild) – 10 (severe)
<code>notes</code>	string	No	Optional free-text observation notes
<code>loggedAt</code>	Date	Yes	UTC timestamp of the observation

Collection: `treatments`

Field	Type	Required	Description
<code>_id</code>	int	Yes	Auto-incremented primary key
<code>name</code>	string	Yes	Unique treatment name (e.g., "Apoquel")
<code>treatmentType</code>	string	Yes	Category: <code>medication</code> , <code>topical</code> , <code>dietary</code> , or <code>therapy</code>
<code>description</code>	string	No	Description of the treatment

Seeded values: Apoquel, Cytopoint Injection, Medicated Shampoo, Grain-Free Diet, Hydrocortisone Cream, Allergy Immunotherapy

Collection: `treatmentLogs`

Field	Type	Required	Description
<code>_id</code>	int	Yes	Auto-incremented primary key
<code>petId</code>	int	Yes	Reference to <code>pets._id</code>
<code>petName</code>	string	Yes	Denormalized pet name (embedded at write time)
<code>treatmentId</code>	int	Yes	Reference to <code>treatments._id</code>
<code>treatmentName</code>	string	Yes	Denormalized treatment name (embedded at write time)
<code>treatmentType</code>	string	Yes	Denormalized treatment type (embedded at write time)
<code>dosage</code>	string	No	Dosage administered (e.g., "16mg tablet")
<code>notes</code>	string	No	Optional administration notes
<code>administeredAt</code>	Date	Yes	UTC timestamp of administration

Collection: **envFactorTypes**

Field	Type	Required	Description
_id	int	Yes	Auto-incremented primary key
name	string	Yes	Unique factor name (e.g., "Pollen Count")
unit	string	Yes	Unit of measurement (e.g., "index", "°F", "%")
description	string	No	Description of what the factor measures

Seeded values: Pollen Count (index, 0–12 scale), Temperature (°F), Humidity (%), Mold Spore Count (index, 0–12 scale)

Collection: **envFactorLogs**

Field	Type	Required	Description
_id	int	Yes	Auto-incremented primary key
petId	int	Yes	Reference to pets._id
petName	string	Yes	Denormalized pet name (embedded at write time)
envFactorTypeId	int	Yes	Reference to envFactorTypes._id
envFactorTypeName	string	Yes	Denormalized factor name (embedded at write time)
envFactorTypeUnit	string	Yes	Denormalized unit (embedded at write time)
value	double	Yes	Numeric reading (e.g., 9.5 for pollen index)
notes	string	No	Optional notes about conditions
loggedAt	Date	Yes	UTC timestamp of the reading

Collection: **llmQueryLogs**

Field	Type	Required	Description
_id	int	Yes	Auto-incremented primary key
userId	int	Yes	Reference to users._id
naturalLanguageQuery	string	Yes	The original question the user typed
generatedSql	string	No	The MongoDB aggregation pipeline JSON generated by the LLM
responseSummary	string	No	Plain-English summary returned to the user
success	boolean	Yes	Whether the query executed successfully
errorMessage	string	No	Error details if success is false

Field	Type	Required	Description
createdAt	Date	Yes	UTC timestamp of the query

Collection: counters

Field	Type	Description
_id	string	Collection name (e.g., "symptomLogs")
seq	int	Current max integer ID for that collection

5. Sample Data

Sample: symptomLogs Documents

```
{
  "_id": 1,
  "petId": 1,
  "petName": "Finn",
  "symptomTypeId": 1,
  "symptomTypeName": "Itching",
  "severity": 3,
  "notes": "Mild scratching after morning walk – spring has arrived",
  "loggedAt": { "$date": "2026-05-01T19:00:00Z" }
}

{
  "_id": 23,
  "petId": 1,
  "petName": "Finn",
  "symptomTypeId": 1,
  "symptomTypeName": "Itching",
  "severity": 9,
  "notes": "Peak severity – applied hydrocortisone, Apoquel not enough",
  "loggedAt": { "$date": "2026-05-13T14:00:00Z" }
}
```

Sample: treatmentLogs Documents

```
{
  "_id": 1,
  "petId": 1,
  "petName": "Finn",
  "treatmentId": 1,
  "treatmentName": "Apoquel",
  "treatmentType": "medication",
  "dosage": "16mg tablet",
}
```

```

    "notes": "Morning dose with food",
    "administeredAt": { "$date": "2026-05-01T13:00:00Z" }
  }

  {
    "_id": 32,
    "petId": 1,
    "petName": "Finn",
    "treatmentId": 2,
    "treatmentName": "Cytopoint Injection",
    "treatmentType": "medication",
    "dosage": "40mg injection",
    "notes": "Monthly Cytopoint – early May dose before pollen peak",
    "administeredAt": { "$date": "2026-05-05T20:00:00Z" }
  }

```

Sample: envFactorLogs Documents

```

{
  "_id": 13,
  "petId": 1,
  "petName": "Finn",
  "envFactorTypeId": 1,
  "envFactorTypeName": "Pollen Count",
  "envFactorTypeUnit": "index",
  "value": 11.8,
  "notes": "Season peak – worst day",
  "loggedAt": { "$date": "2026-05-13T12:00:00Z" }
}

{
  "_id": 45,
  "petId": 1,
  "petName": "Finn",
  "envFactorTypeId": 2,
  "envFactorTypeName": "Temperature",
  "envFactorTypeUnit": "°F",
  "value": 80.0,
  "notes": "Hottest day of month – mold and pollen peak",
  "loggedAt": { "$date": "2026-05-21T12:00:00Z" }
}

```

Sample: llmQueryLogs Document

```

{
  "_id": 1,
  "userId": 1,
  "naturalLanguageQuery": "What are Finn's most common symptoms?",
  "generatedSql": "{$collection\":\"symptomLogs\", \"pipeline\": [{ \"$group\":

```

```
{\"_id\": \"symptomTypeName\", \"count\": {\"sum\": 1}, \"avgSeverity\": {\"avg\": \"severity\"}}, {\"sort\": {\"count\": -1}}, {\"limit\": 10}}\",
  \"responseSummary\": \"Finn's most common symptom is Itching (logged 45 times, average severity 5.2), followed by Paw Licking (28 times, avg 4.8) and Skin Redness (22 times, avg 4.5).\",
  \"success\": true,
  \"errorMessage\": null,
  \"createdAt\": { \"date\": \"2026-05-01T20:15:00Z\" }
}
```

Dataset Summary

The application ships with three seed datasets:

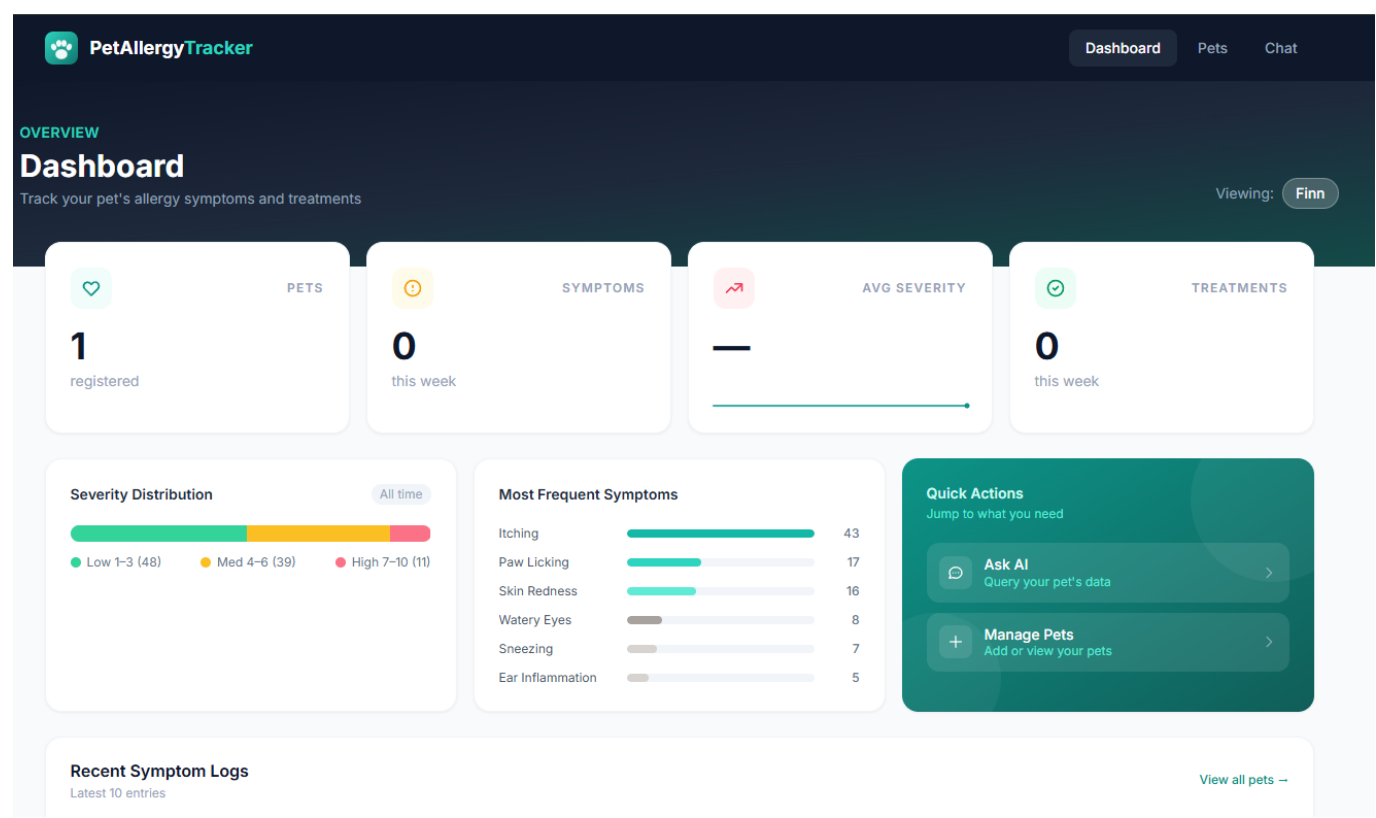
Script	Period	Symptom Logs	Treatment Logs	Env Factor Logs
seed.js	Dec 15, 2025 – Feb 13, 2026	~74	~55	~90
seed_may_2026.js	May 1–31, 2026	71	37	72

May 2026 Story: Peak spring allergy season. Two major flares — May 10–14 (grass pollen peaks at 11.8) and May 19–24 (mixed pollen + mold wave, mold peaks at 8.0). Cytopoint injections given May 5 and May 28. Apoquel administered daily.

6. User Interfaces and Forms

The application is a multi-page web application served at <http://localhost:7070>.

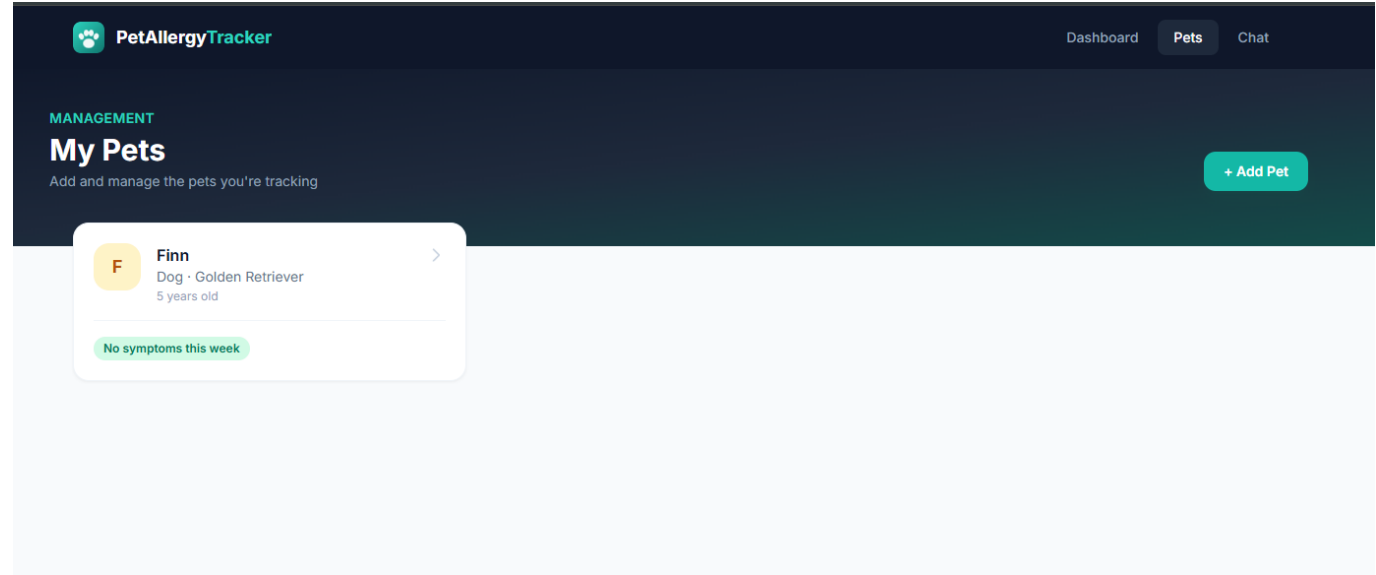
Page 1: Dashboard (/)



The dashboard provides an at-a-glance overview of pet health data:

- Summary cards showing total pets, symptom logs, treatment logs, and average severity
- A severity trend line chart showing symptom severity over time
- A top symptoms bar chart ranked by frequency
- A recent activity feed showing the latest logs across all categories

Page 2: Pets (/pets.html)



Displays all registered pets as cards. Each card shows the pet's name, species, breed, and age. An **Add Pet** button opens a form to register a new pet with fields for name, species, breed, and date of birth.

Page 3: Pet Detail (/pet-detail.html?petId=1)

PetAllergyTracker

Dashboard

Pets

Chat

< Back to Pets

F

Finn

Dog · Golden Retriever · 5 years old

98

symptoms

3.9

avg severity

54

treatments

Symptoms

Treatments

Environment

From

To

mm/dd/yyyy

mm/dd/yyyy

Filter

Clear

+ Log Symptom

DATE	SYMPTOM	SEVERITY	NOTES	
Feb 13, 2026 9:00 AM	Itching	2	Great day	Delete
Feb 12, 2026 2:00 PM	Watery Eyes	2	Eyes clear	Delete
Feb 12, 2026 8:00 AM	Itching	2	Comfortable and happy	Delete
Feb 11, 2026 9:00 AM	Paw Licking	2	Minimal symptoms	Delete
Feb 10, 2026 8:30 AM	Itching	2	Doing great	Delete
Feb 9, 2026 10:00 AM	Skin Redness	2	Skin clear and healthy	Delete
Feb 8, 2026 9:00 AM	Itching	2	Back to normal	Delete

The main data-entry interface for a single pet. Contains three logging forms:

- **Log Symptom** — select symptom type, enter severity (1–10 slider), optional notes, date/time
- **Log Treatment** — select treatment, enter dosage, optional notes, date/time
- **Log Environmental Factor** — select factor type, enter value, optional notes, date/time

Below the forms, three tabbed tables display recent log history for symptoms, treatments, and environmental factors.

Page 4: AI Chat (/chat.html)

PetAllergyTracker

Dashboard

Pets

Chat

What are Finn's most common symptoms?

You

Based on the allergy data for Finn, his most common symptoms include Itching (43 occurrences), followed closely by Skin Redness (16 occurrences) and Paw Licking (17 occurrences). Interestingly, Ear Inflammation appears to be Finn's most severe symptom, with an average severity of 5.2 out of 5. These symptoms suggest that Finn may be experiencing moderate to severe allergy-related discomfort.

Show Pipeline

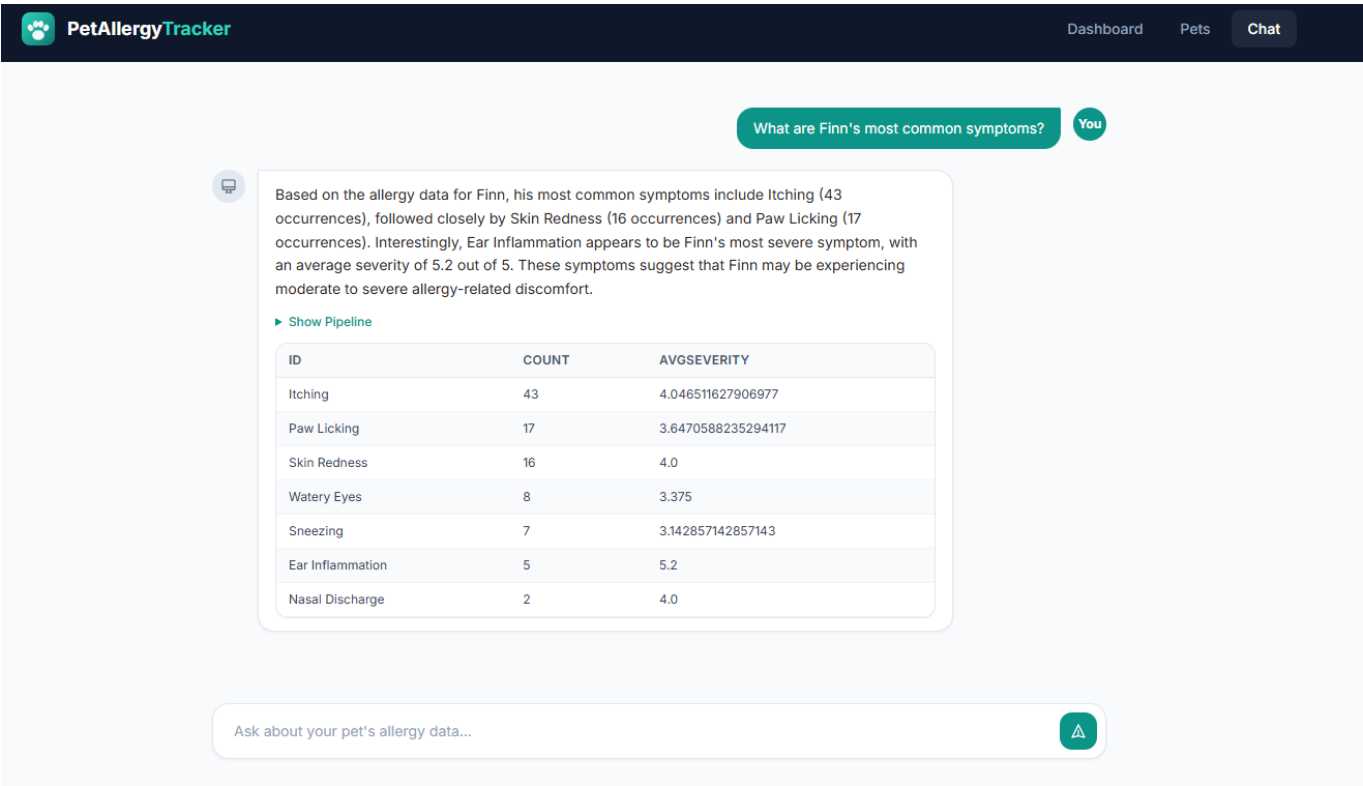
ID	COUNT	AVGSEVERITY
Itching	43	4.046511627906977
Paw Licking	17	3.6470588235294117
Skin Redness	16	4.0
Watery Eyes	8	3.375
Sneezing	7	3.142857142857143
Ear Inflammation	5	5.2
Nasal Discharge	2	4.0

Ask about your pet's allergy data...

12 / 22

A conversational interface where users type natural language questions. The response displays:

- A plain-English summary from the LLM
- A collapsible **Show Pipeline** section revealing the raw MongoDB aggregation pipeline JSON
- A formatted results table with the actual database records returned



Sample interactions:

- "What are Finn's most common symptoms?" → groups and counts symptom logs
- "Show average severity by month" → date-bucket aggregation
- "Is there a correlation between pollen and itching?" → compares envFactorLogs and symptomLogs values over time

7. Source Code

App.java — Application Entry Point

```
public class App {
    public static void main(String[] args) {
        MongoDBDatabase db = DatabaseConfig.getDatabase();

        // DAOs
        PetDao petDao = new PetDao(db);
        SymptomTypeDao symptomTypeDao = new SymptomTypeDao(db);
        SymptomLogDao symptomLogDao = new SymptomLogDao(db);
        TreatmentDao treatmentDao = new TreatmentDao(db);
        TreatmentLogDao treatmentLogDao = new TreatmentLogDao(db);
        EnvFactorTypeDao envFactorTypeDao = new EnvFactorTypeDao(db);
        EnvFactorLogDao envFactorLogDao = new EnvFactorLogDao(db);
    }
}
```

```

LlmQueryLogDao llmQueryLogDao = new LlmQueryLogDao(db);

// Services
OllamaService ollamaService = new OllamaService(db, llmQueryLogDao);

Javalin app = Javalin.create(config -> {
    config.staticFiles.add("/public", Location.CLASSPATH);
    config.http.defaultContentType = "application/json";
    config.jsonMapper(new JavalinJackson().updateMapper(mapper -> {
        mapper.registerModule(new JavaTimeModule());
        mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
    }));
});

petController.registerRoutes(app);
symptomController.registerRoutes(app);
treatmentController.registerRoutes(app);
envFactorController.registerRoutes(app);
chatController.registerRoutes(app);

app.start(7070);
}
}

```

DatabaseConfig.java — MongoDB Connection and Auto-Increment

```

public class DatabaseConfig {

    public static MongoDBDatabase getDatabase() {
        if (mongoClient == null) {
            String uri = env("MONGODB_URI", "mongodb://localhost:27017");
            String dbName = env("MONGODB_DATABASE", "pet_allergy_tracker");
            mongoClient = MongoClient.create(uri);
            database = mongoClient.getDatabase(dbName);
            initIndexes();
            seedData();
        }
        return database;
    }

    // Atomic auto-increment using a counters collection
    public static int getNextId(String collection) {
        Document result = database.getCollection("counters").findOneAndUpdate(
            new Document("_id", collection),
            new Document("$inc", new Document("seq", 1)),
            new
FindOneAndUpdateOptions().upsert(true).returnDocument(ReturnDocument.AFTER)
        );
        return result.getInteger("seq");
    }
}

```

```

private static void initIndexes() {
    database.getCollection("symptomLogs")
        .createIndex(Indexes.ascending("petId", "loggedAt"));
    database.getCollection("treatmentLogs")
        .createIndex(Indexes.ascending("petId", "administeredAt"));
    database.getCollection("envFactorLogs")
        .createIndex(Indexes.ascending("petId", "loggedAt"));
    database.getCollection("pets")
        .createIndex(Indexes.ascending("userId"));
    database.getCollection("llmQueryLogs")
        .createIndex(Indexes.ascending("userId", "createdAt"));
}
}

```

SymptomLogDao.java — Insert with Denormalization

```

public SymptomLog insert(SymptomLog log) {
    int id = DatabaseConfig.getNextId("symptomLogs");
    log.setSymptomLogId(id);
    if (log.getLoggedAt() == null) log.setLoggedAt(Instant.now());

    // Denormalize: embed type name and pet name at write time
    Document typeDoc = symptomTypesCol.find(Filters.eq("_id",
log.getSymptomTypeId())).first();
    String typeName = typeDoc != null ? typeDoc.getString("name") : "";
    log.setSymptomTypeName(typeName);

    Document petDoc = petsCol.find(Filters.eq("_id", log.getPetId())).first();
    String petName = petDoc != null ? petDoc.getString("name") : "";

    Document doc = new Document("_id", id)
        .append("petId", log.getPetId())
        .append("petName", petName)
        .append("symptomTypeId", log.getSymptomTypeId())
        .append("symptomTypeName", typeName)
        .append("severity", log.getSeverity())
        .append("notes", log.getNotes())
        .append("loggedAt", Date.from(log.getLoggedAt()));

    collection.insertOne(doc);
    return log;
}

```

OllamaService.java — LLM Integration (Key Methods)

```
// Validate and safety-check the LLM-generated pipeline JSON
private String validateQuery(String raw) {
    // Strip markdown code fences if present
    String cleaned = raw.replaceAll("(?s)` `[a-z]*\\n?", "").replaceAll("` ``",
    "").trim();

    // Parse with BsonDocument for strict JSON validation
    BsonDocument parsed = BsonDocument.parse(cleaned);

    // Validate collection name against allowlist
    String collection = parsed.getString("collection").getValue();
    if (!ALLOWED_COLLECTIONS.contains(collection)) {
        throw new QueryRejectionException("Collection not allowed: " +
collection);
    }

    // Block write stages ($out, $merge)
    BsonArray pipeline = parsed.getArray("pipeline");
    for (int i = 0; i < pipeline.size(); i++) {
        BsonDocument stage = pipeline.get(i).asDocument();
        if (stage.containsKey("$out") || stage.containsKey("$merge")) {
            throw new QueryRejectionException("Write stages ($out, $merge) are not
permitted");
        }
    }
    return cleaned;
}

// Execute the aggregation pipeline against MongoDB
private List<Map<String, Object>> executeAggregation(String pipelineJson) {
    BsonDocument parsed = BsonDocument.parse(pipelineJson);
    String collectionName = parsed.getString("collection").getValue();
    BsonArray pipelineArray = parsed.getArray("pipeline");

    List<Document> stages = new ArrayList<>();
    for (int i = 0; i < pipelineArray.size(); i++) {
        stages.add(Document.parse(pipelineArray.get(i).asDocument().toJson()));
    }

    List<Map<String, Object>> results = new ArrayList<>();
    db.getCollection(collectionName).aggregate(stages)
        .forEach(doc -> results.add(docToMap(doc)));
    return results;
}
```

ChatController.java — REST Endpoint

```
public class ChatController {
```

```
public void registerRoutes(Javalin app) {
    app.post("/api/chat", this::chat);
    app.get("/api/chat/history", this::chatHistory);
}

private void chat(Context ctx) throws Exception {
    Map<?, ?> body = ctx.bodyAsClass(Map.class);
    String query = (String) body.get("query");

    if (query == null || query.isBlank()) {
        ctx.status(400).json(Map.of("error", "Query is required"));
        return;
    }

    try {
        Map<String, Object> response = ollamaService.processQuery(query,
            DEFAULT_USER_ID);
        ctx.json(response);
    } catch (OllamaService.OllamaUnavailableException e) {
        ctx.status(503).json(Map.of(
            "error", "LLM service unavailable. Make sure Ollama is running.",
            "naturalLanguageQuery", query
        ));
    }
}
```

REST API Endpoints

Method	Path	Description
GET	/api/pets	List all pets
POST	/api/pets	Add a new pet
PUT	/api/pets/:id	Update pet details
GET	/api/pets/:id/symptoms	Get symptom logs for a pet
POST	/api/pets/:id/symptoms	Log a new symptom
GET	/api/pets/:id/treatments	Get treatment logs for a pet
POST	/api/pets/:id/treatments	Log a new treatment
GET	/api/pets/:id/envfactors	Get environmental factor logs
POST	/api/pets/:id/envfactors	Log an environmental factor
POST	/api/chat	Submit a natural language query to the LLM
GET	/api/chat/history	Retrieve past AI chat queries

8. NoSQL Commands

schema.js — Index and Validator Setup

```
// Run with: mongosh pet_allergy_tracker schema.js

// Drop and recreate collections for clean setup
db.symptomLogs.drop();
db.treatmentLogs.drop();
db.envFactorLogs.drop();
db.llmQueryLogs.drop();
db.symptomTypes.drop();
db.treatments.drop();
db.envFactorTypes.drop();
db.pets.drop();
db.users.drop();
db.counters.drop();

// Create compound indexes on primary query patterns
db.symptomLogs.createIndex({ petId: 1, loggedAt: 1 });
db.treatmentLogs.createIndex({ petId: 1, administeredAt: 1 });
db.envFactorLogs.createIndex({ petId: 1, loggedAt: 1 });
db.pets.createIndex({ userId: 1 });
db.llmQueryLogs.createIndex({ userId: 1, createdAt: -1 });

// Add JSON Schema validator to enforce required fields and severity range
db.createCollection("symptomLogs", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["petId", "symptomTypeId", "severity", "loggedAt"],
      properties: {
        petId: { bsonType: "int" },
        symptomTypeId: { bsonType: "int" },
        severity: { bsonType: "int", minimum: 1, maximum: 10,
          description: "Severity must be 1-10" },
        loggedAt: { bsonType: "date" }
      }
    }
  }
});
```

seed.js — Sample Aggregation Queries (mongosh)

Query 1: Most common symptoms, sorted by frequency

```
db.symptomLogs.aggregate([
  { $group: {
    _id: "$symptomTypeName",
```

```
    count: { $sum: 1 },
    avgSeverity: { $avg: "$severity" }
  }},
  { $sort: { count: -1 } }
]);
```

Query 2: Average symptom severity by month

```
db.symptomLogs.aggregate([
  { $group: {
    _id: {
      year: { $year: "$loggedAt" },
      month: { $month: "$loggedAt" }
    },
    avgSeverity: { $avg: "$severity" },
    totalLogs: { $sum: 1 }
  }},
  { $sort: { "_id.year": 1, "_id.month": 1 } }
]);
```

Query 3: All treatments Finn has received, most frequent first

```
db.treatmentLogs.aggregate([
  { $match: { petName: { $regex: "finn", $options: "i" } } },
  { $group: {
    _id: { name: "$treatmentName", type: "$treatmentType" },
    count: { $sum: 1 },
    lastGiven: { $max: "$administeredAt" }
  }},
  { $sort: { count: -1 } }
]);
```

Query 4: Pollen count on days when itching severity was 7 or higher

```
db.symptomLogs.aggregate([
  { $match: { symptomTypeName: "Itching", severity: { $gte: 7 } } },
  { $project: {
    date: { $dateToString: { format: "%Y-%m-%d", date: "$loggedAt" } },
    severity: 1
  }},
  { $sort: { date: 1 } }
]);
```

Query 5: Symptom flare days (severity ≥ 7) count by month

```
db.symptomLogs.aggregate([
  { $match: { severity: { $gte: 7 } } },
  { $group: {
    _id: {
      year: { $year: "$loggedAt" },
      month: { $month: "$loggedAt" }
    },
    flareDays: { $sum: 1 },
    peakSeverity: { $max: "$severity" }
  }},
  { $sort: { "_id.year": 1, "_id.month": 1 } }
]);
```

Query 6: Average pollen count on symptom flare days vs. non-flare days

```
// High-pollen days (pollen > 8)
db.envFactorLogs.aggregate([
  { $match: { envFactorTypeName: "Pollen Count", value: { $gt: 8 } } },
  { $group: { _id: null, avgPollen: { $avg: "$value" }, count: { $sum: 1 } } }
]);
```

Query 7: Total treatment administrations by type

```
db.treatmentLogs.aggregate([
  { $group: {
    _id: "$treatmentType",
    total: { $sum: 1 },
    treatments: { $addToSet: "$treatmentName" }
  }},
  { $sort: { total: -1 } }
]);
```

Query 8: Atomic auto-increment (counters collection)

```
// Get next ID for the symptomLogs collection
db.counters.findOneAndUpdate(
  { _id: "symptomLogs" },
  { $inc: { seq: 1 } },
  { upsert: true, returnDocument: "after" }
);
```

Query 9: Insert a new symptom log

```

db.symptomLogs.insertOne({
  _id: db.counters.findOneAndUpdate(
    { _id: "symptomLogs" },
    { $inc: { seq: 1 } },
    { upsert: true, returnDocument: "after" }
  ).seq,
  petId: 1,
  petName: "Finn",
  symptomTypeId: 1,
  symptomTypeName: "Itching",
  severity: 6,
  notes: "Scratching after afternoon walk",
  loggedAt: new Date()
});

```

Query 10: Find all symptom logs for Finn in May 2026

```

db.symptomLogs.find({
  petId: 1,
  loggedAt: {
    $gte: ISODate("2026-05-01T00:00:00Z"),
    $lte: ISODate("2026-05-31T23:59:59Z")
  }
}).sort({ loggedAt: 1 });

```

Project File Structure

```

project-root/
├── README.md
├── backend/
│   ├── pom.xml
│   └── src/main/
│       ├── java/com/petallergy/
│       │   ├── App.java
│       │   ├── config/DatabaseConfig.java
│       │   ├── model/          (Pet, SymptomLog, TreatmentLog, EnvFactorLog,
LlmQueryLog, ...)
│       │   ├── dao/           (PetDao, SymptomLogDao, TreatmentLogDao,
EnvFactorLogDao, ...)
│       │   ├── controller/   (PetController, SymptomController,
TreatmentController,
│       │   │   │               EnvFactorController, ChatController)
│       │   └── service/OllamaService.java
│       └── resources/
│           ├── public/        (index.html, pets.html, pet-detail.html,
chat.html,
│           │               css/app.css, js/api.js, js/dashboard.js, ...)

```

```
├── db/
│   ├── schema.js      (mongosh: indexes + validators)
│   ├── seed.js        (mongosh: Dec 2025 - Feb 2026 log data)
│   └── seed_may_2026.js (mongosh: May 2026 spring flare data)
└── logback.xml
├── docs/
│   ├── CS5600DESC_23405_Group2.pdf
│   └── FinalReport_23405_Group2.md
```

Submitted for CS 5600 — Advanced Database Systems, Spring 2026 Sam Cashion | 700619652 | CRN 23405 | Group 2